

Committed Composite Approaches

How blinded commitments close the partition leak in the hierarchical TSP proofs

Supervisor review note - 2026-06-24

Purpose. This note explains the two committed variants implemented in the project: committed-sort and committed-product. It is written to be readable without the full project context. The focus is:

- what the committed variants prove;
- how they differ from the plain variants;
- why the plain variants leak information;
- how commitments plus a private blinding factor close that leak computationally, even when the hidden domain is small;
- where the mechanism appears in the Noir code.

The implementation paths used in this note are the cleaned export names: `export/circuits/composite_committed_sort_*` and `export/circuits/composite_committed_product_*`. The experimental tree has the same circuits under `circuits/hierarchical_segment_c*` and `circuits/hierarchical_glue_c*`.

1. Background: what is being decomposed

The base statement is the TSP decision statement:

> I know a Hamiltonian cycle over N nodes whose total cost is at most the public threshold $> T$, and every edge cost is read from the cost matrix committed by the public Poseidon2 > Merkle root `root`.

The flat proof proves this in one circuit. The composite proofs split the cycle into K consecutive segments of size $M = \frac{N}{K}$. Each segment proof checks the internal path inside one segment. A separate glue proof checks that the segment proofs assemble into one global cycle:

1. The K segment node sets cover exactly $\{0, \dots, N - 1\}$.
2. The boundary edge from each segment to the next is bound to the same Merkle root.
3. Internal segment costs plus boundary costs are at most T .
4. The independently generated proofs are tied together by public inputs that the verifier cross-checks.

The decomposition gives parallelism and lower per-prover memory. It also creates a new privacy problem: separate proofs need shared values to be stitched together, and if those values are public plaintext, they reveal the partition of the tour.

2. Plain variants and their leak

The plain variants leave the stitching values directly on the public surface.

2.1. Plain-sort

In plain-sort, each segment publishes its sorted node set, endpoints, partial cost, and root. The segment keeps the visit order private, but the verifier sees exactly which nodes are grouped together.

```
// export/circuits/composite_plain_sort_segment/src/main.nr
fn main(
  cycle_segment: [u32; M],
  edge_costs: [u64; M - 1],
  siblings: [Field; (M - 1) * DEPTH],
  path_bits: [bool; (M - 1) * DEPTH],

  sorted_nodes: pub [u32; M],
  start_node: pub u32,
  end_node: pub u32,
  partial_cost: pub u64,
```

```

    root: pub Field,
) { ... }

```

The glue then publishes the concatenation of all segment node sets and sorts it in-circuit:

```

// export/circuits/composite_plain_sort_glue/src/main.nr
fn main(
  boundary_costs: [u64; K],
  boundary_siblings: [Field; K * DEPTH],
  boundary_path_bits: [bool; K * DEPTH],

  all_sorted_nodes: pub [u32; N],
  starts: pub [u32; K],
  ends: pub [u32; K],
  partial_costs: pub [u64; K],
  threshold: pub u64,
  root: pub Field,
) {
  let sorted = all_sorted_nodes.sort_via(|a: u32, b: u32| a <= b);
  for i in 0..N {
    assert(sorted[i] == i as u32, "partition does not cover {0..N-1}");
  }
  ...
}

```

This is sound and simple, but it fully discloses the partition. For example, at $N = 8, K = 2$, the verifier sees the two 4-node sets and the endpoints. Only the interior orderings remain hidden.

2.2. Plain-product

plain-product replaces each public node list by a field fingerprint:

$$P_i = \prod_{v \text{ in segment } i} (X + v).$$

The glue checks:

$$\prod_i P_i = \prod_{j=0}^{N-1} (X + j).$$

This is much smaller than a public list and distributes the permutation work across the segments. However, the public values are still deterministic functions of small-domain data:

```

// export/circuits/composite_plain_product_segment/src/main.nr
P_i: pub Field,
h_in_i: pub Field,
h_out_i: pub Field,
c: pub Field,
X: pub Field,

```

The grand-product value P_i is a confirmation oracle for the segment set: for small N , an observer can enumerate candidate subsets S of size M and test whether $\prod_{v \in S} (X + v) = P_i$. The chain anchors h_in_i and h_out_i similarly help confirm candidate orders. This is not cryptographic hiding; it is hiding behind the cost of a search. That is especially weak on small benchmark domains, and it remains dangerous whenever the adversary already has a plausible candidate partition.

3. The committed idea

The committed variants keep the same segment/glue architecture, but change what crosses the public boundary.

Instead of publishing the segment values themselves, segment i publishes one opaque commitment:

$$C_i = \text{fold}(r_i, \text{values}_i).$$

The fold used in the implementation is an iterated two-input Poseidon2 hash:

$$\text{acc}_0 = r_i, \quad \text{acc}_{t+1} = H(\text{acc}_t, v_t), \quad C_i = \text{acc}_\ell.$$

Here r_i is a fresh private blinding scalar for segment i . The values folded into the commitment depend on the mechanism:

- committed-sort folds the segment nodes in cycle order and the partial internal cost.
- committed-product folds the product fingerprint, hash-chain anchors, endpoints, and partial internal cost.

The verifier never sees the folded values. It sees only C_i .

The glue receives the same values and the same r_i as private witness. Inside the glue circuit, it recomputes the fold and asserts that the result is the public C_i . Therefore:

1. The segment proof says: “my hidden segment values open to this public C_i .”
2. The glue proof says: “the hidden values I use for global stitching also open to this same public C_i .”
3. The verifier checks that the segment’s public C_i equals the glue’s public $C_{is}[i]$.

If the commitment is binding, the segment and the glue are forced to use the same hidden values. If the commitment is hiding, the public C_i does not reveal those values.

4. Why the blinding factor matters

Without r_i , a hash commitment to small-domain data is just a deterministic fingerprint. A verifier could enumerate every possible segment and recompute the hash until it matches. That is the same dictionary attack as in plain-product, only with a different hash.

The private blinding factor changes the attack. To confirm a guessed segment, the verifier must also find a private r_i such that:

$$\text{fold}(r_i, \text{guess}) = C_i.$$

The builder samples r_i from `/dev/urandom` with 128 bits of entropy and passes it only as private witness. In the random-oracle model for Poseidon2, this turns the check into a preimage search over the blinding space. The hidden message domain can be tiny, such as $N = 8$, but the confirmation attack still has to pay the blinding cost.

This is the important distinction:

- Plain surface: “try candidate segment -> compare public value.”
- Blinded commitment: “try candidate segment -> still cannot compare without also finding r_i .”

So the committed variants give computational hiding of the partition. This is not information-theoretic hiding: an unbounded attacker could enumerate the blinding space, and the claim rests on Poseidon2 behaving as a random oracle / preimage-resistant hash. But it removes the low-cost brute-force oracle that exists in the plain variants.

5. committed-sort

committed-sort is the direct privacy upgrade of plain-sort.

5.1. Public/private change

Plain-sort public surface:

- per segment: sorted_nodes, start_node, end_node, partial_cost, root;
- glue: all_sorted_nodes, starts, ends, partial_costs, threshold, root.

Committed-sort public surface:

- per segment: root, C_i ;
- glue: root, threshold, C_{is} .

The actual nodes, endpoints, and partial costs move into the witness. The glue still performs the same exact sort-based partition check, but now over private, commitment-bound witness values.

5.2. Segment commitment in Noir

The committed-sort segment computes the true internal cost from private edge witnesses, then folds the private segment nodes and that cost into C_i :

```
// export/circuits/composite_committed_sort_segment/src/main.nr
fn main(
  cycle_segment: [u32; M],
  edge_costs: [u64; M - 1],
  siblings: [Field; (M - 1) * DEPTH],
  path_bits: [bool; (M - 1) * DEPTH],
  r: Field,

  root: pub Field,
  C_i: pub Field,
) {
  ...
  let partial_cost: u64 = sum;

  let mut acc: Field = r;
  for j in 0..M {
    acc = Poseidon2::hash([acc, cycle_segment[j] as Field], 2);
  }
  acc = Poseidon2::hash([acc, partial_cost as Field], 2);
  assert(acc == C_i, "commitment mismatch");
}
```

Intuition: the segment proof no longer says “these are my public nodes.” It says “I know private nodes and a private blinding scalar that open this public commitment.”

5.3. Glue opening and global checks in Noir

The committed-sort glue receives all segment nodes and the corresponding openings as witness. It first recomputes the commitments, then runs the ordinary global checks on the hidden values.

```
// export/circuits/composite_committed_sort_glue/src/main.nr
fn main(
  boundary_costs: [u64; K],
  boundary_siblings: [Field; K * DEPTH],
  boundary_path_bits: [bool; K * DEPTH],
  all_nodes: [u32; N],
  partial_costs: [u64; K],
  r_is: [Field; K],

  root: pub Field,
  threshold: pub u64,
  C_is: pub [Field; K],
) {
  for i in 0..K {
    let mut acc: Field = r_is[i];
    for j in 0..M {
      acc = Poseidon2::hash([acc, all_nodes[i * M + j] as Field], 2);
    }
    acc = Poseidon2::hash([acc, partial_costs[i] as Field], 2);
    assert(acc == C_is[i], "commitment mismatch");
  }

  let sorted = all_nodes.sort_via(|a: u32, b: u32| a <= b);
  for i in 0..N {
    assert(sorted[i] == i as u32, "partition does not cover {0..N-1}");
  }

  ...
  assert(total <= threshold, "cycle cost exceeds threshold");
}
```

The sort check is still exact: if the private `all_nodes` contain a duplicate or miss a node, the equality to `[0..N-1]` fails. The difference is that the verifier no longer sees the list being sorted.

6. committed-product

committed-product is the privacy upgrade of plain-product.

It keeps the product mechanism, so it still uses:

- per-segment grand products P_i ;
- a Poseidon2 hash chain over the full cycle;
- a Fiat-Shamir challenge $X = H(c)$;
- the global product equality for partition coverage.

But the per-segment values are no longer public. They are folded into C_i .

6.1. Public/private change

Plain-product public surface includes:

- starts, ends, partial_costs;
- P_{is} ;
- chain anchors h_{ins}, h_{outs} ;
- chain terminal c ;
- challenge X ;
- root and threshold.

Committed-product public surface includes only:

- segment proof: root, X, C_i ;
- glue proof: root, threshold, X, C_{is} .

The challenge X remains public because all segment proofs and the glue must agree on the same challenge. The chain terminal c and the per-segment anchors move into the witness. The public X is a residual whole-cycle-level value, but the per-segment partition oracle is removed.

6.2. Segment commitment in Noir

The segment computes the product and chain values privately, then folds those values into the public commitment:

```
// export/circuits/composite_committed_product_segment/src/main.nr
let mut h: Field = h_in_i;
for j in 0..M {
  h = Poseidon2::hash([h, cycle_segment[j] as Field], 2);
}
let h_out: Field = h;

let mut prod: Field = 1;
for j in 0..M {
  prod = prod * (X + cycle_segment[j] as Field);
}
let p_i: Field = prod;

let mut acc: Field = r;
acc = Poseidon2::hash([acc, p_i], 2);
acc = Poseidon2::hash([acc, h_in_i], 2);
acc = Poseidon2::hash([acc, h_out], 2);
acc = Poseidon2::hash([acc, start_node as Field], 2);
acc = Poseidon2::hash([acc, end_node as Field], 2);
acc = Poseidon2::hash([acc, partial_cost as Field], 2);
assert(acc == C_i, "commitment mismatch");
```

This is the exact place where the plain-product confirmation oracles are hidden. The values `p_i`, `h_in_i`, `h_out`, endpoints, and `partial_cost` still exist because the glue needs them, but they are no longer exposed as public inputs.

6.3. Glue opening, chain, and product partition check in Noir

The committed-product glue first binds its witnessed values to the public commitments. Then it runs the same product-based global checks that plain-product used on public values.

```
// export/circuits/composite_committed_product_glue/src/main.nr
for i in 0..K {
  let mut acc: Field = r_is[i];
  acc = Poseidon2::hash([acc, P_is[i]], 2);
  acc = Poseidon2::hash([acc, h_ins[i]], 2);
  acc = Poseidon2::hash([acc, h_outs[i]], 2);
  acc = Poseidon2::hash([acc, starts[i] as Field], 2);
  acc = Poseidon2::hash([acc, ends[i] as Field], 2);
  acc = Poseidon2::hash([acc, partial_costs[i] as Field], 2);
  assert(acc == C_is[i], "commitment mismatch");
}

assert(h_ins[0] == 0, "chain must start from 0");
for i in 0..(K - 1) {
  assert(h_ins[i + 1] == h_outs[i], "chain not continuous");
}
assert(h_outs[K - 1] == c, "chain does not terminate at c");

let x_expected: Field = Poseidon2::hash([c], 1);
assert(X == x_expected, "X != Poseidon2(c)");

let mut lhs: Field = 1;
for i in 0..K {
  lhs = lhs * P_is[i];
}
let mut rhs: Field = 1;
for j in 0..N {
  rhs = rhs * (X + j as Field);
}
assert(lhs == rhs, "grand-product partition mismatch");
```

The product check is probabilistic, not exact. At a challenge X fixed after the cycle is committed through the hash chain, a wrong multiset passes with Schwartz-Zippel probability at most about $\frac{N}{|F|}$, plus the random-oracle assumptions used for Fiat-Shamir. That is the same soundness class as plain-product; the commitment changes privacy, not the product argument's probabilistic nature.

7. How independent proofs are tied together

The composite variants produce $K + 1$ independent UltraHonk proofs: K segment proofs and one glue proof. A valid proof only binds one circuit to its own public inputs. Therefore the verifier still must cross-check public values across proofs.

For committed-product, the cross-check collapses to root equality, challenge equality, and commitment equality:

```
# export/pipeline/verify_composite_committed_product.py
for shared in ("root", "X"):
    for i, s in enumerate(subs):
        if s[shared] != glue[shared]:
            errors.append(f"{shared} mismatch")

for i, s in enumerate(subs):
    if glue["C_is"][i] != s["C_i"]:
        errors.append(f"C_i mismatch at segment {i}")
```

For committed-sort there is no X , so the verifier checks only the shared root and the commitment equality:

```
# export/pipeline/verify_composite_committed_sort.py
for i, s in enumerate(subs):
    if s["root"] != glue["root"]:
        errors.append("root mismatch")

for i, s in enumerate(subs):
    if glue["C_is"][i] != s["C_i"]:
        errors.append(f"C_i mismatch at segment {i}")
```

This is the “commit-and-prove” pattern in this implementation: the verifier stitches proofs together using opaque commitments, while the circuits prove that the opaque values open to the data needed for the local and global checks.

8. Witness generation and the blinding scalar

The Rust builder computes the same fold off-circuit when it writes the `Prover.toml` files. This is not trusted for soundness; it only prepares the honest witness. Soundness comes from the Noir assertions above.

```
// export/pipeline/merkle_builder/src/main.rs
fn commit_fold(r: FieldElement, vals: &[FieldElement]) -> FieldElement {
    let mut acc = r;
    for &v in vals {
        acc = poseidon2_compress(acc, v);
    }
    acc
}

fn random_field() -> FieldElement {
    use std::io::Read;
    let mut buf = [0u8; 16];
    std::fs::File::open("/dev/urandom")
        .and_then(|mut f| f.read_exact(&mut buf))
        .expect("read /dev/urandom for blinding");
    FieldElement::from(u128::from_le_bytes(buf))
}
```

For committed-sort, the builder folds the segment nodes and cost:

```
let r = random_field();
let mut fold_vals: Vec<FieldElement> =
    cycle_segment.iter().map(|&v| FieldElement::from(v as u128)).collect();
fold_vals.push(FieldElement::from(partial_cost as u128));
let c_i = commit_fold(r, &fold_vals);
```

For committed-product, it folds the product/checking values:

```
let r = random_field();
let c_i = commit_fold(
    r,
    &[
        p_i,
        h_in,
        h_out,
        FieldElement::from(start_node as u128),
        FieldElement::from(end_node as u128),
        FieldElement::from(partial_cost as u128),
    ],
);
```

The important implementation invariant is that the segment and glue receive the same opening r_i privately, and the verifier receives only C_i .

9. What security property each mechanism buys

Completeness. An honest prover can generate the segment openings, the segment proofs, the glue opening, and the verifier cross-checks pass.

Binding. The same public C_i cannot be opened to different segment values without breaking the Poseidon2 fold as a collision / second-preimage target. This is what prevents a malicious prover from using one hidden segment in the segment proof and a different hidden segment in the glue proof.

Hiding. Because r_i is private and high entropy, C_i does not act as a dictionary oracle for small-domain segment guesses. This is computational hiding under the Poseidon2 random-oracle or preimage-resistance assumption.

Partition correctness. The commitment itself does not prove the partition. It only hides and binds the values. The partition is still enforced by:

- committed-sort: exact `sort(all_nodes) == [0..N-1]`;
- committed-product: probabilistic grand-product equality at Fiat-Shamir challenge X .

Matrix correctness. All variants still bind edge costs to the same Poseidon2 Merkle root by checking both the Merkle path and the leaf index from $* N + to$. The committed variants do not change the matrix representation.

SNARK layer. UltraHonk proves knowledge of witnesses satisfying these circuits. Its own knowledge soundness and zero-knowledge assumptions are common to all variants and are not what distinguishes committed from plain.

10. What is still leaked

The committed variants close the partition oracle, but they are not identical to the flat or recursive privacy surface.

- The verifier still learns K , the number of segments, because there are K commitments and K segment proofs.
- The verifier sees root and threshold, as in the flat committed baseline.
- In committed-product, the verifier also sees the shared challenge X . The chain terminal c and anchors are hidden, so the per-segment oracle is removed, but X remains a public value tied to the full-cycle product argument.
- Proof metadata and timings are outside the circuit-level hiding claim.

Therefore the privacy ladder is:

1. Plain variants: partition exposed or confirmable from public deterministic values.
2. Committed variants: partition hidden computationally behind blinded commitments.
3. Recursive/flat endpoint: partition structurally absent from the public interface.

11. Summary comparison

Variant	Public stitch values	Partition check	Privacy status
plain-sort	sorted_nodes, endpoints, costs	Exact sort in glue	Partition disclosed
plain-product	P_i, anchors, endpoints, costs	Grand product + FS	Search-based hiding only; small-domain oracle
committed-sort	C_i commitments	Exact sort over witness	Computational hiding via secret r_i
committed-product	C_i commitments plus public X	Grand product + FS over witness	Computational hiding via secret r_i ; product soundness probabilistic

The main point is that commitments do not replace the TSP checks. They move the stitching values from public plaintext into private witness, while preserving a public handle that independent proofs can be cross-checked against. The blinding factor is what prevents that public handle from becoming a deterministic brute-force oracle on the small node domain.